

Exercise 1: Setting Up JUnit

Code

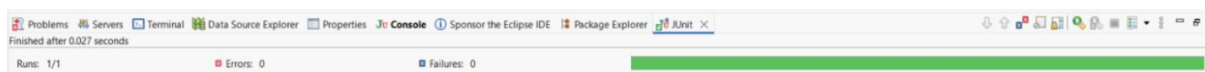
Calculator.java

```
package com.tapasvi.junit;  
  
public class Calculator {  
    public int addNumbers(int firstValue, int secondValue) {  
        return firstValue + secondValue;  
    }  
}
```

CalculatorTest.java

```
package com.tapasvi.junit;  
  
import static org.junit.Assert.assertEquals;  
  
import org.junit.Test;  
  
public class CalculatorTest {  
    @Test  
    public void verifyAddition() {  
        Calculator calculator = new Calculator();  
        int result = calculator.addNumbers(15, 25);  
        assertEquals(40, result);  
    }  
}
```

Output



Result

The JUnit framework was configured successfully. The Calculator class was tested using JUnit 4, and the addition operation completed successfully without any errors or failures.

Exercise 3: Assertions in JUnit

Code

```
package com.tapasvi.junit;

import static org.junit.Assert.*;

import org.junit.Test;

public class AssertionsTest {

    @Test

    public void checkAssertions() {

        assertEquals(8, 4 + 4);

        assertTrue(20 > 10);

        assertFalse(8 < 5);

        String message = null;

        assertNull(message);

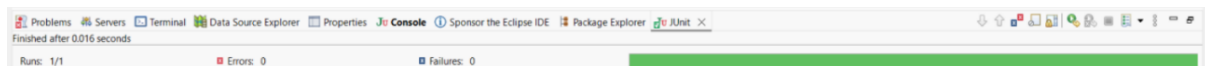
        Object data = new Object();

        assertNotNull(data);

    }

}
```

Output



Result

Various JUnit assertion methods, including `assertEquals()`, `assertTrue()`, `assertFalse()`, `nullNull()`, and `assertNotNull()`, were executed successfully. All test cases passed without any errors or failures.

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Code

CalculatorAAA.java

```
package com.tapasvi.junit;

public class CalculatorAAA {

    public int multiplyNumbers(int firstValue, int secondValue) {

        return firstValue * secondValue;

    }

}
```

CalculatorAAATest.java

```
package com.tapasvi.junit;

import static org.junit.Assert.assertEquals;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class CalculatorAAATest {

    private CalculatorAAA calculator;

    @Before

    public void setUp() {

        calculator = new CalculatorAAA();

        System.out.println("Initialization completed");

    }

    @After

    public void tearDown() {

        calculator = null;

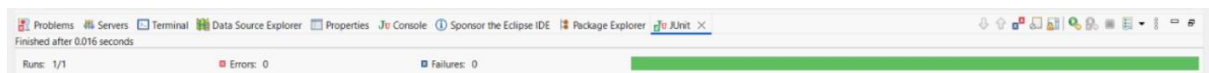
        System.out.println("Resources released");

    }

    @Test
```

```
public void verifyMultiplication() {  
    // Arrange  
    int firstNumber = 7;  
    int secondNumber = 4;  
    // Act  
    int result = calculator.multiplyNumbers(firstNumber, secondNumber);  
    // Assert  
    assertEquals(28, result);  
}  
}
```

Output



Result

The Arrange-Act-Assert (AAA) pattern was implemented successfully using JUnit. The @Before method initialized the required object before execution, and the @After method released the resources after the test completed. The multiplication test executed successfully without any errors or failures.

Mockito Exercises

Exercise 1: Mocking and Stubbing Using Mockito

Code

ExternalApi.java

```
package com.tapasvi.junit;

public interface ExternalApi {

    String fetchData();

}
```

DataService.java

```
package com.tapasvi.junit;

public class DataService {

    private ExternalApi api;

    public DataService(ExternalApi api) {

        this.api = api;

    }

    public String getData() {

        return api.fetchData();

    }

}
```

DataServiceTest.java

```
package com.tapasvi.junit;

import static org.junit.Assert.assertEquals;

import static org.mockito.Mockito.*;

import org.junit.Test;

public class DataServiceTest {

    @Test

    public void verifyMocking() {

        ExternalApi apiMock = mock(ExternalApi.class);

        when(apiMock.fetchData()).thenReturn("Demo Response");

    }

}
```

```
DataService service = new DataService(apiMock);

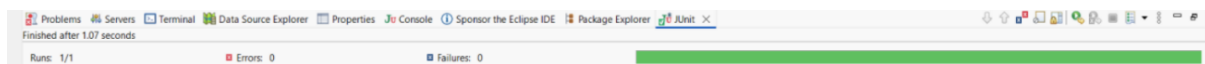
String output = service.getData();

assertEquals("Demo Response", output);

}

}
```

Output



Result

Mockito was used to create a mock object for the external API. The expected response was successfully stubbed, and the service returned the mocked data correctly. The test completed successfully without any errors or failures.

Exercise 2: Verifying Interactions Using Mockito

Code

InteractionTest.java

```
package com.tapasvi.junit;

import static org.mockito.Mockito.*;

import org.junit.Test;

public class InteractionTest {

    @Test

    public void verifyMethodInvocation() {

        ExternalApi apiMock = mock(ExternalApi.class);

        when(apiMock.fetchData()).thenReturn("Verified");

        DataService service = new DataService(apiMock);

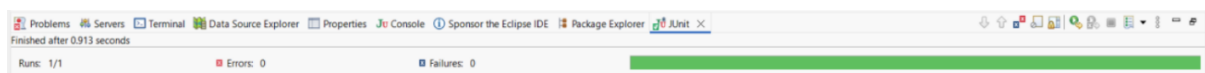
        service.getData();

        verify(apiMock).fetchData();

    }

}
```

Output



Result

Mockito successfully verified that the `fetchData()` method of the mocked object was invoked during test execution. The interaction test completed successfully without any errors or failures.